AI API Reverse-Engineering Platform: Zero-Execution AST Analysis with Grounded LLM Reasoning for Automated API Documentation

Nagesh Pawar [1 a], Guide Name [2 b]

[1] *Department of Computer Science and Engineering, Your Institution Name, City, Country*

[2] *Department of Computer Science and Engineering, Your Institution Name, City, Country*

your_email@institution.edu, guide_email@institution.edu

Keywords: API Reverse Engineering, Static Analysis, Abstract Syntax Tree, OpenAPI, Large Language Models, Design Science, DevSecOps.

Abstract: Backend repositories are often inherited with little or no API documentation, and recovering the contract of an API by hand is slow and error-prone. This paper follows a design science approach to build and evaluate the AI API Reverse-Engineering Platform, which ingests a ZIP archive or a public GitHub repository and recovers the API surface without executing the untrusted code. A deterministic abstract syntax tree (AST) engine resolves routes, mount prefixes, middleware, parameters, status codes and ORM schemas, and a grounded reasoning layer based on large language models adds business intent while labelling each statement as detected or inferred with a file and line citation. A single normalised model drives OpenAPI 3.0 specifications, Postman collections, CI/CD pipeline flows, ER diagrams, security audits, Jest and Supertest suites and an SSRF-protected playground. Evaluated through three research questions on an e-commerce and authentication demonstration backend, the engine discovered all routes (RC = 100%), executed no repository code and produced all seven artifact types in one run. The study is limited to one backend and one framework; broader benchmarks and a user study are planned.

1 INTRODUCTION

1.1 Background and Motivation

Web APIs are the integration layer of modern software, and REST [1] remains the dominant style for exposing them [2]. In microservice-oriented systems [3]–[5] the number of endpoints grows quickly and routes are added, renamed and re-mounted continuously, while the documents that should describe them are updated late or never. Empirical studies of API documentation report incomplete, ambiguous and outdated content as recurrent problems [6], [7], practitioner surveys find that documentation is valued yet frequently out of date [8], and the cost and benefit of documentation have been studied systematically [9]. Poor documentation is also among the barriers reported for newcomers to open-source projects [10].

For a developer who inherits an undocumented backend, the source code is the only authoritative statement of the API contract, yet recovering it means reading nested router definitions, middleware chains and controllers spread over many files. The consequences are felt downstream: front-end teams guess request and response shapes, testers write collections by hand, and automated REST test generators, which typically consume an OpenAPI description [11]–[14], cannot start without an accurate one.

Existing automation covers only part of the problem. Annotation-based generators need framework-specific metadata that developers must add beforehand [15], and dynamic approaches need a running system and representative inputs [16]. Large language models (LLMs) summarise code fluently [17], [18] but hallucinate when they are not anchored to verifiable evidence [19], [20]. The safety of the analysis host is a further concern: a repository is untrusted input, and tools that install dependencies or start the application in order to inspect it execute arbitrary code taken from that repository.

1.2 Research Questions and Contributions

This paper treats the problem as a design science task [21], [22]: an artifact, the AI API Reverse-Engineering Platform, is built and then evaluated against explicit research questions (RQs): • **RQ1 (completeness):** how completely does a zero-execution AST engine recover the route surface of an Express backend? • **RQ2 (safety):** can an untrusted repository be analysed without executing its code on the analysis host? • **RQ3 (artifact coverage):** can the documentation, specification, graph, security and test artifacts be generated from a single knowledge model in one run? A fourth question, on the effect of the platform on developer onboarding time, requires a user study and is deferred to Section 8.

The contributions of the paper are: (1) a zero-execution ingestion and AST pipeline that recovers routes, mount prefixes, middleware chains, parameters, status codes and ORM schemas from Node.js and Express projects written in JavaScript or TypeScript; (2) a grounding discipline in which compiler facts are labelled DETECTED and language-model statements are labelled INFERRED with file and line citations, which limits the effect of hallucination; (3) a canonical knowledge model from which OpenAPI, Postman, CI/CD pipeline flows, ER diagrams, security findings, test suites, an SSRF-protected playground and a citation-backed chat are generated in one dashboard; and (4) a structured evaluation of completeness, safety and artifact coverage on a demonstration backend.

The paper is organised as follows. Section 2 reviews related work, Section 3 describes the research methodology, Section 4 presents the system design, Section 5 the implementation, Section 6 the evaluation, Section 7 the threats to validity and Section 8 concludes.

2 RELATED WORK

2.1 API Documentation and Specification Recovery

Inferring API properties from code is an established topic. Robillard et al. [16] survey static and dynamic techniques that infer properties such as the call ordering of library APIs and show that program analysis can recover constraints without developer annotations. Their focus is in-process libraries, not network-facing endpoints, whose contract is formed by verbs, path templates assembled across files, middleware and payload shapes. Wittern et al. [23] analyse JavaScript clients statically to check web API requests against specifications; the work shows the value of static reasoning for web APIs but operates on the consumer side and presupposes an existing specification. On the server side, Lercher et al. [15] generate OpenAPI descriptions from Java source code and compare their accuracy with annotation-driven tools such as springdoc-openapi. This supports the premise that specifications can be derived from code, but the approach is specific to the Java ecosystem. The need for accurate specifications is reinforced by REST testing research. RESTler [11], EvoMaster [12] and the tools compared by Kim et al. [13] work from API descriptions, a recent survey [14] organises the field around them, and KAT [24] adds LLM-based test generation that exploits dependencies between operations. Maalej and Robillard [7] provide a taxonomy of knowledge types in API reference documentation against which generated documentation can be assessed.

2.2 Static Analysis of JavaScript

Static analysis of JavaScript is difficult because of the dynamic features of the language. Type analysis for JavaScript was an early response [25], later work shows that frameworks and libraries complicate analysis further [26], and a survey of dynamic analysis and test generation for JavaScript [27] documents the trade-offs between the two families of techniques. The engine presented here therefore does not aim at whole-program soundness. It exploits the fact that Express route registration follows a small set of syntactic idioms, so that a syntax-directed traversal of the abstract syntax tree [28], [29] suffices for the common cases, and it marks what it cannot resolve statically. The theory of such analyses is covered by Nielson et al. [30]. Practical experience also matters. Johnson et al. [31] found that developers avoid static analysis tools when warnings are false or hard to act upon, and Sadowski et al. [32] report that analysis at scale succeeded at Google because findings were actionable and integrated into developer workflows. This motivates the DETECTED and INFERRED labels of the platform, which tell the reader how much each statement can be trusted.

2.3 Static and Dynamic Analysis in Security-Oriented Reverse Engineering

Reverse engineering is widely practised in security, where the choice between static and dynamic analysis is a recurring theme. Javed et al. [33] review 236 papers on binary code analysis and devote one of their six areas to evasion and anti-analysis strategies and their countermeasures. Evasion is common: in 117,270 Android apps, Suo et al. [34] found that almost all benign (99.6%) and malicious (97.0%) apps implement at least one category of anti-runtime-analysis technique, which can lead to inaccurate analysis results. Norby et al. [35] measure the performance overhead of six categories of anti-debugging techniques on Windows and Linux, and Padhy et al. [36] show how binding execution to a machine through DRM fingerprinting and thread-local-storage callbacks thwarts sandbox, virtual-machine and debugger-based analysis. Static approaches remain effective in this setting. Kurniawan et al. [37] reach 97% accuracy in detecting Android malware from statically extracted features, and Bart■omiejczyk et al. [38] report that permission-based static analysis still detects SMS-stealing malware under several obfuscation strategies, with a detection rate of almost 70%. Mori■ et al. [39] conclude that hybrid analysis that integrates static, dynamic and simulated testing improves detection reliability, but they carry out the dynamic work in controlled environments, which illustrates the cost of executing untrusted code safely. These studies concern binaries and malware rather than API source code. We use them only as evidence for a design principle, namely that the analysis of untrusted artifacts should not depend on running them, and apply that principle to source-level API recovery.

2.4 LLMs for Software Engineering and Documentation

Transformer-based language models [40] pre-trained on text and code [17], [41], [42] are now applied across the software engineering lifecycle [18], [43]. For documentation, RepoAgent [44] generates repository-level documentation with an LLM. For reverse engineering, Hu et al. [45] systematise LLM-based work and report that retrieval and grounding improve factual accuracy while hallucination and weak reproducibility remain open issues, and Siala and Lano [46] and Boronat and Mustafa [47] explore LLMs in model-driven reverse engineering. Hallucination is documented extensively [19], [20], retrieval-augmented generation [48], [49] is the main mitigation, LLM-written code has been shown to contain security weaknesses [50], and repository-level benchmarks such as SWE-bench [51] show that tasks requiring an understanding of whole repositories remain hard for language models.

2.5 Summary and Research Gap

Table 1 positions the platform against representative work. Three observations motivate the design. First, each contribution solves one sub-problem, such as specification generation, documentation or testing, whereas developers need the whole set of artifacts in one place. Second, specification generators are tied to one technology stack and, when LLMs are used, the output is rarely separated into verified facts and model conjecture. Third, the safety of the analysis host is seldom treated as a design requirement, although repository analysis necessarily handles untrusted code.

Table 1: Positioning of this work against related approaches.

Work	Analysis	Runs code?	Main output
Robillard et al. [16]	Survey: static /dynamic	Partly	Library API properties
Wittern et al. [23]	Static, JS clients	No	Request checks
Lercher et al. [15]	Static, Java source	No	OpenAPI description
Luo et al. [44]	LLM, repository level	No	Code docs
Le et al. [24]	LLM, spec driven	Yes (live API)	API test cases
This work	AST + grounded LLM	No	Specs, graphs, tests, audits

3 RESEARCH METHODOLOGY

3.1 Research Design

The work follows the design science paradigm [21], in which knowledge is gained by building and evaluating an artifact that addresses a relevant problem. The process is structured according to the six activities of the design science research methodology of Peffers et al. [22]; Table 2 maps each activity to the part of this paper in which it is reported. The evaluation follows the case-study guidelines of Runeson and Höst [52], with the platform applied to a demonstration backend as the unit of analysis, and the research questions are refined with the Goal–Question–Metric approach [53].

Table 2: Design science activities and where they are reported.

Activity	Application in this work	Section
1. Problem	Undocumented backends, unsafe analysis, LLM hallucination	1, 3.2
2. Objectives	Five objectives O1 to O5	3.2
3. Design	Four-stage pipeline, AST engine, grounding	4, 5
4. Demonstration	Analysis of an e-commerce and authentication API	6.1
5. Evaluation	RQ1 to RQ3 with defined metrics	6.2
6. Communication	This paper	all

3.2 Problem Statement and Objectives

Four recurring challenges motivate the work. (i) Knowledge loss and onboarding friction: when authors leave, undocumented routing logic becomes tribal knowledge and newcomers must trace nested router.use() calls, middleware gates and controllers manually. (ii) Contract mismatch and QA overhead: front-end and mobile teams guess request and response shapes, and testers write Postman collections and curl commands by hand. (iii) Hidden architectural dependencies: nothing shows which route reaches which controller, service or database model. (iv) Security exposure: state-changing endpoints (POST, PUT, DELETE) frequently lack authentication middleware [54], and dynamic analysis tools run the untrusted application, for example with npm start, which exposes the analysis host.

The objectives of the platform are therefore to: • **(O1)** build an extensible static AST parser for Node.js and Express that extracts routes, handlers and parameter schemas; • **(O2)** construct an ORM analyzer that extracts database models, fields, keys and relations; • **(O3)** synthesise standards-compliant OpenAPI 3.0 specifications and Postman collections; • **(O4)** develop a static security scanner that flags broken authentication, missing input validation and injection-prone patterns; • **(O5)** deploy a conversational assistant that answers codebase questions with exact file and line citations.

The current version covers Node.js and Express REST architectures in JavaScript and TypeScript, Prisma, Mongoose and Sequelize models, safe ZIP upload and public GitHub cloning, an interactive React Flow graph with a Mermaid ER diagram, Jest and Supertest synthesis and a guarded API playground. Analyzers for Fastify, NestJS, Django, FastAPI, Spring Boot and Laravel, and real-time Git webhook synchronisation, are outside the scope of this version.

3.3 Goal, Questions and Metrics

Following the Goal–Question–Metric approach [53], the goal is to analyse the platform for the purpose of evaluation with respect to completeness, safety and artifact coverage, from the viewpoint of a developer who inherits an undocumented Express backend. Table 3 lists the research questions with their metrics and the evidence collected.

Table 3: Research questions, metrics and evidence.

RQ	Metric	Evidence collected
RQ1	Route coverage RC (Eq. 2)	Engine output vs. manual route set
RQ2	Host executions HE	Inspection of pipeline and run observation
RQ3	Artifact types (out of 7)	One analysis run; dashboard outputs checked
RQ4	Onboarding time, INFERRED accuracy	Planned user study (future work)

4 SYSTEM DESIGN

4.1 Architecture Overview

Figure 1 shows the four-stage pipeline. Stage 1 ingests and sanitises the repository, Stage 2 extracts deterministic facts with an AST engine, Stage 3 enriches them with a grounded reasoning layer, and Stage 4 renders all artifacts from the resulting normalised model. The ordering is deliberate: the language model never sees unverified input as truth and never writes into the set of detected facts.

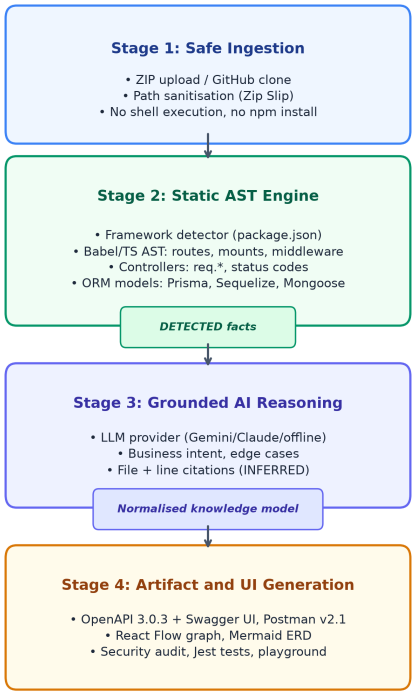


Figure 1: Four-stage architecture of the platform.

4.2 Safe Ingestion

The platform accepts a ZIP archive or the URL of a public GitHub repository. Archives are extracted with explicit path sanitisation: the resolved path of every entry must remain inside the workspace directory, which blocks Zip Slip attacks in which entries such as ../../x escape the extraction directory [55]. Public repositories are cloned into the workspace and treated exactly like an extracted archive. No step invokes a package manager, a build tool or a shell on repository content; files are only read as text and parsed. Consequently, install scripts, post-install hooks and start-up code of the uploaded project cannot run on the analysis host.

4.3 Static AST Engine

The engine parses every JavaScript and TypeScript file with the Babel parser [29] and traverses the trees with the Babel traversal library. Four analyses run over the resulting syntax trees.

4.3.1 Framework Detection

A detector inspects the package.json dependencies and file-level heuristics, for example imports of express [56] and calls to Router(), to select an analyzer. At present Express is supported; the analyzer interface is designed so that Fastify, NestJS or non-JavaScript frameworks can be added as plugins.

4.3.2 Route and Mount Resolution

Express applications assemble URLs across files: a router declares relative paths and a parent mounts it under a prefix with app.use. The full path of a route r is therefore the normalised concatenation of all mount prefixes on the way from the application entry module to r, followed by the path of the route itself (Equation (1)).

$$P(r) = norm(m_1 \cdot m_2 \cdot \ldots \cdot m_k \cdot p_r) \qquad (1)$$

Here mi are the mount prefixes, pr is the route path declared in the router, · denotes string concatenation and norm collapses duplicate and trailing slashes. Algorithm 1 performs this resolution recursively. Starting at the entry module with an empty prefix, it records every verb call (get, post, put, patch, delete) together with its middleware list and handler reference and, for every mount call, resolves the imported child router through the import graph and recurses with the extended prefix. A visited set keyed by module and prefix prevents infinite recursion on cyclic imports.

```
Data: module M, prefix π (initially empty),
visited set V
Result: route records R
if (M, π) ∈ V then return ∅;
add (M, π) to V; R ← ∅;
foreach call expression c in AST(M) do
  if c is router.VERB(path, mw…, h) then
    add (VERB, norm(π · path), mw, h) to R;
  else if c is app.use(prefix, child) then
    M′ ← resolve child via imports;
    R ← R ∪ ResolveRoutes(M′, norm(π ·
prefix), V);
  end
end
return R
```

Algorithm 1: Recursive route and mount resolution.

Listing 1 illustrates the result for a single mounted route. Every field of the record originates from syntax and is therefore labelled DETECTED.

```
app.use('/api/v1', orders);
// orders.js
router.post('/orders',
requireAuth, create);
// => POST /api/v1/orders
// middleware: requireAuth
// handler: create [DETECTED]
```

Listing 1: Mounted route and the extracted record.

4.3.3 Controller Analysis

Handlers are resolved from inline functions or imported controller modules. For each handler the analyzer records destructuring and member access on req.params, req.query and req.body, which yields parameter names and their location, and collects the arguments of res.status and res.sendStatus calls, which yields the status codes that the handler can return. Because these facts come directly from syntax they are classified DETECTED. Types and constraints that cannot be read from syntax are not guessed by the engine; they are left to the reasoning layer, where they are labelled INFERRED.

4.3.4 ORM Model Extraction

The database analyzer reads Prisma schema files, Sequelize model definitions and Mongoose schemas and extracts entities, fields, keys and relations into a common representation. This representation feeds the Mermaid [57] ER diagram and gives the CI/CD pipeline flow its edges from endpoints to the data entities they touch.

4.4 Grounded AI Reasoning

The reasoning layer receives the normalised model rather than raw repository text. A provider abstraction supports Gemini, OpenAI and Claude and falls back to an offline heuristic engine that needs no API key. The layer adds business purpose, plain-language descriptions, error explanations and edge cases. Two rules limit hallucination. First, deterministic facts such as paths, verbs, parameters, status codes and middleware are copied from Stage 2 and tagged DETECTED; the model can neither add nor remove them. Second, everything the model contributes is tagged INFERRED and carries a citation to the file and line that supports it. The same discipline governs the chat assistant which, in the spirit of retrieval-augmented generation [48], [49], answers questions from the model and source excerpts and returns the file and line of each supporting item.

4.5 Artifact Generation

Table 4 lists the artifacts. All generators read the same knowledge model, so they stay mutually consistent, and re-running the analysis after a code change regenerates every artifact, which supports living documentation. The OpenAPI output follows version 3.0.3 of the specification [58] and the Postman output follows collection format v2.1.0 [59]; the CI/CD pipeline flow is rendered with React Flow [60].

Table 4: Artifacts generated from the knowledge model.

Artifact	Format	Derived from
API docs	OpenAPI 3.0.3, Swagger UI	Routes, params, statuses
Postman	Collection v2.1.0 JSON	Routes, parameters
CI/CD pipeline	React Flow	Route, controller, model links
Database	Mermaid ER	ORM models
Security	Findings list	Middleware chains, handlers
Test suite	Jest and Supertest	Routes, observed status codes
Playground	HTTP console (SSRF guard)	OpenAPI model

4.6 Security Scanner, Tests and Playground

The static security scanner flags broken authentication, that is state-changing routes without authentication middleware in their chain, missing input validation, and injection-prone patterns. These categories correspond to entries of the OWASP API Security Top 10 [54]. The test generator emits one Jest [61] and Supertest case per route with the observed status codes as expected outcomes. The playground lets the user send requests to a chosen target from the dashboard; SSRF guardrails prevent the console from being used to reach internal network resources.

4.7 Data Flow

Figure 2 presents the data flow in data flow diagram notation [62], [63]: rectangles denote external entities, circles processes and open rectangles data stores. At Level 0 the platform is a single process between the developer, who supplies a ZIP file or a Git URL, and the dashboard that returns the results. Figure 2 decomposes it at Level 1 into ingestion (1.0), framework detection (2.0), AST analysis (3.0, comprising route extraction 3.1, controller analysis 3.2 and database analysis 3.3), AI reasoning (4.0) and artifact generation (5.0). Data store D1 holds the normalised knowledge model and D2 to D5 hold the generated artifacts.

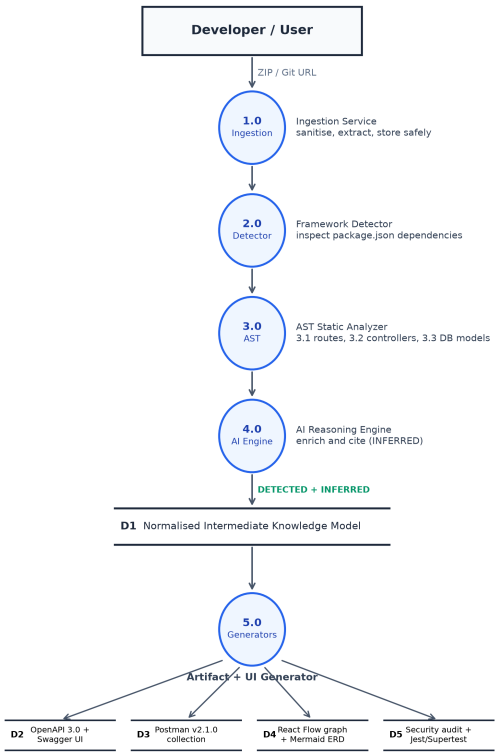


Figure 2: Level-1 data flow diagram of the platform.

5 IMPLEMENTATION

Table 5 summarises the technology stack. The client is a React 18 single-page application built with Vite and TypeScript; the server is written in TypeScript on Node.js 18 or later (tested on Node 24 LTS) with Express. Persistence uses PostgreSQL through a connection pool and falls back automatically to a local JSON store, so that the platform runs on a laptop without set-up. The whole stack consists of free and open-source software and the offline reasoning provider needs no paid key, so the running cost is zero. A machine with an Intel Core i5, AMD Ryzen 5 or Apple Silicon processor, 8 GB of RAM (16 GB for large workspaces) and 2 GB of storage is sufficient.

Table 5: Technology stack.

Layer	Technologies
Client	React 18, Vite, TypeScript, Tailwind, React Flow, swagger-ui-react, mermaid.js, Router
Server	Node.js, Express, TypeScript, adm-zip, yaml, axios
AST engine	Babel parser, traverse and types
Storage	PostgreSQL with JSON-file fallback
AI layer	Pluggable: Gemini, OpenAI, Claude, offline heuristics

The system was developed in about 200 engineering hours over eight weeks in six phases: foundation and architecture; ingestion and the AST engine; database analysis and the intermediate model; generators and visualisations; security, tests, playground and AI chat; and evaluation with a demonstration.

6 EVALUATION

6.1 Experimental Setup

The platform was applied to a demonstration backend, a multi-layered e-commerce and authentication REST API written for Node.js and Express and distributed with the project (examples/demo-api). The ground truth is the set of routes obtained by manually reading the source, including nested mounts. The analysis consists of a single run on the unmodified source. For RQ1 the route coverage is defined in Equation (2):

$$RC = |R_{det} \cap R_{gt}| / |R_{gt}| \times 100\% \qquad (2)$$

where Rdet is the set of routes reported by the engine and Rgt the ground truth. For RQ2 the host-execution count HE is the number of processes spawned or packages installed on repository content; the target value is zero. For RQ3 the number of artifact types of Table 4 that the run produced is counted.

6.2 Results

Table 6 summarises the outcome. RQ1: the engine discovered every route of the ground truth, so RC = 100%, including nested routes for which the prefix concatenation of Equation (1) produced the full path. RQ2: the analysis consisted of file reads and AST parsing only, and no repository code was executed; HE = 0. RQ3: the single run produced all seven artifact types of Table 4, which were exposed in the dashboard. Parameters and status codes observed in handlers appear in the specification as DETECTED, whereas explanatory text is labelled INFERRED with citations.

Table 6: Summary of results on the demonstration backend.

RQ	Metric	Result

RQ1	Route coverage RC	100%
RQ2	Host executions HE	0
RQ3	Artifact types generated	7 of 7
RQ4	Onboarding time, INFERRED accuracy	Not yet evaluated

6.3 Discussion

The results indicate that a syntax-directed engine is sufficient to recover the route surface of a conventionally structured Express backend without annotations, which is the main difference from annotation-based generators [15], and that zero-execution analysis is achievable in practice. The separation of DETECTED and INFERRED statements lets a reviewer concentrate verification effort on the inferred part, which addresses the concern that analysis output is hard to act upon [31]. The expectation that onboarding time falls markedly, with a project target of more than 60%, remains a hypothesis (RQ4) until a user study has been carried out.

7 THREATS TO VALIDITY

The threats are discussed along the categories of Wohlin et al. [64]. Construct validity: RC measures route discovery only; the correctness of parameters, schemas and descriptions is not covered, and the ground truth was derived manually by the authors, so errors in it would affect RC. Internal validity: the DETECTED part is deterministic, but the INFERRED part depends on the language model and may vary between runs and providers. External validity: a single backend authored within the project and a single framework were used; repositories that register routes dynamically, for instance in loops, from computed paths or through plugins, cannot be fully resolved by static analysis and are untested. Conclusion validity: with one case no statistical inference is possible and the results are descriptive. There is also no baseline comparison with annotation-based generators or LLM-only approaches. A benchmark of open-source Express projects with ground-truth routes and a controlled study with developers are the next steps.

8 CONCLUSIONS AND FUTURE WORK

This paper presented the AI API Reverse-Engineering Platform, developed and evaluated as a design science artifact. It recovers the API surface of a backend repository by static AST analysis, enriches it with a grounded reasoning layer and generates documentation, specifications, graphs, security findings and tests from one knowledge model without executing the analysed code. On the demonstration backend it achieved complete route coverage (RQ1), executed no repository code (RQ2) and produced all seven artifact types in one run (RQ3). The explicit separation of DETECTED facts from INFERRED statements, with file and line citations, is the central mechanism against hallucination.

Future work covers plugin analyzers for Fastify, NestJS, Django, FastAPI, Spring Boot and Laravel, webhook-driven re-analysis for continuous documentation, an evaluation on a benchmark of open-source repositories with a quantitative comparison to baselines, and a user study that measures onboarding time (RQ4) and the rate of hallucinated statements.

REFERENCES

[1] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[2] L. Richardson, M. Amundsen, and S. Ruby, RESTful Web APIs. Sebastopol, CA, USA: O’Reilly Media, 2013.

[3] S. Newman, Building Microservices, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2021.

[4] J. Soldani, D. A. Tamburri, and W.-J. van den Heuvel, “The pains and gains of microservices: A systematic grey literature review,” J. Syst. Softw., vol. 146, pp. 215–232, 2018.

[5] P. Di Francesco, P. Lago, and I. Malavolta, “Migrating towards microservice architectures: An industrial survey,” in Proc. IEEE Int. Conf. Softw. Archit. (ICSA), 2018, pp. 29–39.

[6] G. Uddin and M. P. Robillard, “How API documentation fails,” IEEE Softw., vol. 32, no. 4, pp. 68–75, Jul./Aug. 2015.

[7] W. Maalej and M. P. Robillard, “Patterns of knowledge in API reference documentation,” IEEE Trans. Softw.

Eng., vol. 39, no. 9, pp. 1264–1282, Sep. 2013.

[8] A. Forward and T. C. Lethbridge, “The relevance of software documentation, tools and technologies: A survey,” in Proc. ACM Symp. Document Eng. (DocEng), 2002, pp. 26–33.

[9] J. Zhi, V. Garousi-Yusifo■lu, B. Sun, G. Garousi, S. Shahnewaz, and G. Ruhe, “Cost, benefits and quality of software development documentation: A systematic mapping,” J. Syst. Softw., vol. 99, pp. 175–198, 2015.

[10] I. Steinmacher, M. A. G. Silva, M. A. Gerosa, and D. F. Redmiles, “A systematic literature review on the barriers faced by newcomers to open source software projects,” Inf. Softw. Technol., vol. 59, pp. 67–85, 2015.

[11] V. Atlidakis, P. Godefroid, and M. Polishchuk, “RESTler: Stateful REST API fuzzing,” in Proc. IEEE/ACM 41st Int. Conf. Softw. Eng. (ICSE), 2019, pp. 748–758.

[12] A. Arcuri, “RESTful API automated test case generation with EvoMaster,” ACM Trans. Softw. Eng. Methodol., vol. 28, no. 1, pp. 3:1–3:37, 2019.

[13] M. Kim, Q. Xin, S. Sinha, and A. Orso, “Automated test generation for REST APIs: No time to rest yet,” in Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA), 2022, pp. 289–301.

[14] A. Golmohammadi, M. Zhang, and A. Arcuri, “Testing RESTful APIs: A survey,” ACM Trans. Softw. Eng. Methodol., vol. 33, no. 1, pp. 27:1–27:41, 2023.

[15] A. Lercher, C. Macho, C. Bauer, and M. Pinzger, “Generating accurate OpenAPI descriptions from Java source code,” arXiv:2410.23873, 2024.

[16] M. P. Robillard, E. Bodden, D. Kawrykow, M. Mezini, and T. Ratchford, “Automated API property inference techniques,” IEEE Trans. Softw. Eng., vol. 39, no. 5, pp. 613–637, May 2013.

[17] M. Chen et al., “Evaluating large language models trained on code,” arXiv:2107.03374, 2021.

[18] X. Hou et al., “Large language models for software engineering: A systematic literature review,” ACM Trans. Softw. Eng. Methodol., vol. 33, no. 8, pp. 1–79, 2024.

[19] Z. Ji et al., “Survey of hallucination in natural language generation,” ACM Comput. Surv., vol. 55, no. 12, pp. 248:1–248:38, 2023.

[20] L. Huang et al., “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” ACM Trans. Inf. Syst., vol. 43, no. 2, pp. 1–55, 2025.

[21] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” MIS Quart., vol. 28, no. 1, pp. 75–105, 2004.

[22] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” J. Manage. Inf. Syst., vol. 24, no. 3, pp. 45–77, 2007.

[23] E. Wittern et al., “Statically checking web API requests in JavaScript,” in Proc. IEEE/ACM 39th Int. Conf. Softw. Eng. (ICSE), 2017, pp. 244–254.

[24] T. Le, T. Tran, D. Cao, V. Le, T. N. Nguyen, and V. Nguyen, “KAT: Dependency-aware automated API testing with large language models,” in Proc. IEEE Conf. Softw. Testing, Verification Validation (ICST), 2024, pp. 82–92.

[25] S. H. Jensen, A. Møller, and P. Thiemann, “Type analysis for JavaScript,” in Proc. Int. Static Anal. Symp. (SAS), 2009, pp. 238–255.

[26] M. Madsen, B. Livshits, and M. Fanning, “Practical static analysis of JavaScript applications in the presence of frameworks and libraries,” in Proc. 9th Joint Meeting Found. Softw. Eng. (ESEC/FSE), 2013, pp. 499–509.

[27] E. Andreasen et al., “A survey of dynamic analysis and test generation for JavaScript,” ACM Comput. Surv., vol. 50, no. 5, pp. 66:1–66:36, 2017.

[28] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[29] Babel Team, “Babel: The compiler for next generation JavaScript,” 2024. [Online]. Available: https://babeljs.io

[30] F. Nielson, H. R. Nielson, and C. Hankin, Principles of Program Analysis. Berlin, Germany: Springer, 1999.

[31] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, “Why don’t software developers use static analysis tools to find bugs?” in Proc. 35th Int. Conf. Softw. Eng. (ICSE), 2013, pp. 672–681.

[32] C. Sadowski, E. Aftandilian, A. Eagle, L. Miller-Cushon, and C. Jaspan, “Lessons from building static analysis tools at Google,” Commun. ACM, vol. 61, no. 4, pp. 58–66, 2018.

[33] H. Javed, F. Ali, B. Shah, and D. Kwak, “Binary code analysis for cybersecurity: A systematic review of forensic techniques in vulnerability detection and anti-evasion strategies,” IEEE Access, vol. 13, pp. 167139–167164, 2025.

[34] D. Suo, L. Xue, L. Yu, R. Tan, W. Huang, and G. Sun, “ARAP: Demystifying anti runtime analysis code in Android apps,” IEEE Trans. Softw. Eng., vol. 51, no. 10, pp. 2787–2803, Oct. 2025.

[35] A. Norby, B. P. Rimal, and B. Brizendine, “Measurement of anti-debugging techniques on Windows and Linux for Intel x86-64,” IEEE Access,

vol. 13, pp. 46568–46583, 2025.

[36] S. T. Padhy, P. B. Shanthi, G. Bijur, and S. Hemalatha, “A threat-informed approach to malware evasion using DRM and TLS callbacks,” IEEE Access, vol. 13, pp. 155195–155202, 2025.

[37] F. Kurniawan, D. Stiawan, D. Antoni, M. Y. Idris, and R. Budiarto, “A robust and efficient machine learning framework for early detection of Android malware,” IEEE Access, vol. 13, pp. 127183–127220, 2025.

[38] M. Bart■omiejczyk, I. El Fray, and F. Kamoun, “Enhancing two-factor authentication security by analyzing SMS OTP-interception in Android malware,” IEEE Access, vol. 13, pp. 185926–185942, 2025.

[39] Z. Mori■, V. Daki■, and A. Kapulica, “An in-depth exploration of malware persistence: Reverse engineering, defense, and simulation,” IEEE Access, vol. 14, pp. 37384–37397, 2026.

[40] A. Vaswani et al., “Attention is all you need,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, 2017, pp. 5998–6008.

[41] Z. Feng et al., “CodeBERT: A pre-trained model for programming and natural languages,” in Findings Assoc. Comput. Linguistics: EMNLP, 2020, pp. 1536–1547.

[42] T. B. Brown et al., “Language models are few-shot learners,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, 2020, pp. 1877–1901.

[43] A. Fan et al., “Large language models for software engineering: Survey and open problems,” in Proc. IEEE/ACM Int. Conf. Softw. Eng.: Future Softw. Eng. (ICSE-FoSE), 2023, pp. 31–53.

[44] Q. Luo et al., “RepoAgent: An LLM-powered open-source framework for repository-level code documentation generation,” arXiv:2402.16667, 2024.

[45] X. Hu, Z. Fu, S. Xie, S. H. H. Ding, and P. Charland, “SoK: Potentials and challenges of large language models for reverse engineering,” arXiv:2509.21821, 2025.

[46] H. A. Siala and K. Lano, “Towards using LLMs in the reverse engineering of software systems to object constraint language,” in Proc. IEEE Int. Conf. Softw. Anal., Evol. Reengineering (SANER), 2025, pp. 1–6.

[47] A. Boronat and J. Mustafa, “MDRE-LLM: A tool for analyzing and applying LLMs in software reverse engineering,” in Proc. IEEE Int. Conf. Softw. Anal., Evol. Reengineering (SANER), 2025, pp. 850–854.

[48] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[49] Y. Gao et al., “Retrieval-augmented generation for large language models: A survey,” arXiv:2312.10997, 2023.

[50] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, “Asleep at the keyboard? Assessing the security of GitHub Copilot’s code contributions,” in Proc. IEEE Symp. Secur. Privacy (SP), 2022, pp. 754–768.

[51] C. E. Jimenez et al., “SWE-bench: Can language models resolve real-world GitHub issues?” in Proc. Int. Conf. Learn. Represent. (ICLR), 2024.

[52] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” Empirical Softw. Eng., vol. 14, no. 2, pp. 131–164, 2009.

[53] V. R. Basili, G. Caldiera, and H. D. Rombach, “The goal question metric approach,” in Encyclopedia of Software Engineering, vol. 1. New York, NY, USA: Wiley, 1994, pp. 528–532.

[54] OWASP Foundation, “OWASP API security top 10 – 2023,” 2023. [Online]. Available: https://owasp.org/API-Security/

[55] Snyk, “Zip Slip vulnerability,” 2018. [Online]. Available: https://security.snyk.io/research/zip-slip-vulnerability

[56] OpenJS Foundation, “Express: Fast, unopinionated, minimalist web framework for Node.js,” 2024. [Online]. Available: https://expressjs.com

[57] Mermaid contributors, “Mermaid: Diagramming and charting tool,” 2024. [Online]. Available: https://mermaid.js.org

[58] OpenAPI Initiative, “OpenAPI Specification v3.0.3,” 2021. [Online]. Available: https://spec.openapis.org/oas/v3.0.3

[59] Postman, “Collection format v2.1.0,” 2021. [Online]. Available: https://schema.postman.com/

[60] xyflow, “React Flow: A library for building node-based user interfaces,” 2024. [Online]. Available: https://reactflow.dev

[61] OpenJS Foundation, “Jest: Delightful JavaScript testing,” 2024. [Online]. Available: https://jestjs.io

[62] T. DeMarco, Structured Analysis and System Specification. New York, NY, USA: Yourdon Press, 1978.

[63] C. Gane and T. Sarson, Structured Systems Analysis: Tools and Techniques. Englewood Cliffs, NJ, USA: Prentice-Hall, 1979.

[64] C. Wohlin et al., Experimentation in Software Engineering. Berlin, Germany: Springer, 2012.